Does the Apple M6 Use the `BC.cond` Branch Hint?

A microbenchmark study of FEAT_HBC on Super, Performance and Efficiency cores

David Sheffield

October 2026

Abstract

The Apple M6 (Mac mini, Mac18,5, macOS 27.0.1) reports `FEAT_HBC` (hinted conditional branches), and both Apple clang and upstream LLVM enable `+hbc` for M6-class targets. Neither compiler emits `BC.cond`, and upstream LLVM has no code-generation path for it, with or without profile-guided optimization. Paired microbenchmarks that differ only in `B.cond` versus `BC.cond` show no robust timing difference on any of the three M6 core types. The small effects that appear near the branch-target-buffer capacity knee change sign with code placement and do not survive a layout-swap control. The same holds for large-footprint (up to 4 MiB) and fully cold code, which overflows the direction predictor even though every branch has a fixed direction, and for a predictor-pollution test. Every apparent effect, up to 47%, follows code placement and vanishes under a randomized multi-placement control. The M6 appears to execute `BC.cond` as an ordinary conditional branch.

1 Background

FEAT_HBC (Armv8.8/v9.3) adds `BC.cond`, a conditional branch carrying a hint that it "will behave very consistently and is very unlikely to change direction." An implementation may use the hint, for example to save predictor storage, or ignore it.

On the M6, `hw.optional.arm.FEAT_HBC` is 1. Apple clang 21 with `-mcpu=apple-m6` and upstream clang 24 (`main`) with `-mcpu=apple-m5` both pass `+hbc`. However, SPEC CPU2026 binaries built by either compiler contain zero `BC.cond` instructions (for example, `cc1` has about 300,000 `B.cond` and no `BC.cond`). In upstream LLVM the instruction is defined only for the assembler (`def BCcc : BranchCond<1, "bc">, Requires<[HasHBC]>`), and no instruction-selection, branch-relaxation or profile-guided pass references it.

2 Method

A generator (`gen_hbc.py`) emits every kernel twice, identical except for the branch mnemonic. Loop control uses `sub`/`cbnz`, so the condition flags belong to the branch under test. The kernels are:

- **single**: one never-taken branch per iteration.
- **fp**N: $N \in \{1\text{K}, 4\text{K}, 16\text{K}, 64\text{K}\}$ static never-taken branches, each followed by an `add`.
- **tk**N: N static always-taken branches, each skipping one `add`; every branch needs a branch-target-buffer entry.
- **alt** and **p8**: an alternating branch and a branch taken once every 8 iterations. Both are learnable from history, so a core that trusted the hint could mispredict them.
- **random50** and **biased95**: a data-dependent branch on a 64 KiB buffer of 50/50 and 95/5 random bytes.

Cycles and instructions come from the process's fixed counters (`proc_pid_rusage`, `RUSAGE_INFO_V6`). Configurable PMU events, such as mispredict counts, are unavailable because `kpc_set_config` returns `EPERM`

on macOS 27, even for Instruments. Each pair is warmed up, then run interleaved `b/bc`; the final numbers use the multi-placement control described below.

macOS has no core-affinity API, so core type is steered by occupying faster cores with fixed-priority (non-timeshare, user-interactive) register-only spinner threads. Running with 0 spinners places the job on a Super core, 2 spinners on a Performance core, and 6 on an Efficiency core. Placement is verified on every run from the Super-core cycle share (`ri_pcycles/ri_cycles`) and from the clock frequency (cycles over CPU time).

Table 1: Core placement achieved per run.

Core type	Spinners	Super-core cycles	Clock (GHz)
Super	0	99.9%	4.78
Performance	2	0.0%	4.51
Efficiency	6	0.0%	3.36

3 Results

Table 2 gives the `BC/B` cycle ratio for each kernel and core type; 1.000 means no difference. Each cell pools 64 code placements: 32 copies of each variant at random offsets, for each of two independent placement seeds, measured over 15 shuffled rounds (the multi-placement control of Section 5). The entry is the geometric mean of the per-placement ratios, with a 95% confidence interval.

Table 2: `BC.cond` / `B.cond` cycles per iteration: geometric mean over 64 random placements [95% CI].

Kernel	Super	Performance	Efficiency
single	1.000 [1.000, 1.001]	1.000 [0.996, 1.004]	1.003 [0.997, 1.009]
fp1K	1.000 [1.000, 1.000]	1.000 [1.000, 1.000]	1.000 [1.000, 1.000]
fp4K	1.000 [1.000, 1.000]	1.000 [1.000, 1.000]	1.000 [1.000, 1.000]
fp16K	1.000 [1.000, 1.000]	1.000 [1.000, 1.000]	1.000 [1.000, 1.000]
fp64K	1.000 [0.999, 1.001]	0.996 [0.992, 1.000]	1.004 [1.000, 1.008]
tk1K	1.006 [0.992, 1.021]	1.001 [0.997, 1.005]	1.000 [0.999, 1.000]
tk4K	1.005 [0.991, 1.019]	1.000 [0.998, 1.001]	1.002 [0.993, 1.012]
tk16K	1.001 [1.000, 1.003]	1.000 [0.999, 1.000]	1.002 [0.999, 1.006]
tk64K	1.001 [0.999, 1.002]	1.000 [0.998, 1.002]	0.995 [0.972, 1.019]
alt	0.979 [0.930, 1.030]	1.000 [1.000, 1.000]	1.000 [1.000, 1.000]
p8	1.000 [1.000, 1.000]	1.000 [1.000, 1.000]	1.006 [0.990, 1.021]
random50	1.008 [0.983, 1.034]	1.005 [0.989, 1.022]	0.998 [0.989, 1.007]
biased95	1.006 [0.982, 1.031]	1.002 [0.978, 1.026]	1.004 [0.980, 1.028]

Of the 39 intervals, 37 contain 1.000, and most are within ±0.5%. The two exceptions are borderline (Performance fp64K, 0.996, $t = -2.2$; Super p8, $t = +2.0$), about what 39 tests at the 95% level produce by chance, and they point in opposite directions. The widest intervals come from placement-dependent bimodality, not from the hint. For example, Super-core alt runs at 1.0 cycle per iteration at most placements but at 2.0 at a few (four `B` copies and two `BC` copies), the same rate as Performance and Efficiency cores.

An initial single-placement pass (medians of 5 runs) showed apparent effects of up to 25%: tk1K 0.931 on Super and 1.120 on Efficiency cores, random50 0.756 on Performance cores, and fp64K 0.963 and 1.037. None survives the multi-placement control. A two-layout swap had already cast doubt on tk1K: on Super cores the ratio moved from 0.974 to 0.989 when the `B` and `BC` addresses were exchanged.

The alternating and period-8 kernels are the strongest evidence that the hint is ignored. They are the opposite of "consistent," yet `BC` costs the same as `B` on every core type.

4 Large-footprint and cold code

If the hint has any value, it should show where predictor state is scarce or missing: code whose branch footprint exceeds the branch-target buffer and direction predictor, or code that runs cold. A second generator (`gen_cold.py`) emits straight-line regions in which every branch has a fixed direction, so every branch is "consistent":

- **mix**: random 50/50 fixed directions, one `add` per branch (8 B/branch);
- **tk** / **nt**: all taken / all not taken (8 B/branch);
- **sparse**: random fixed directions, five `add`s per branch (24 B/branch).

Sizes run from 4K to 512K branches (32 KiB to 4 MiB of code; 3 MiB for sparse), beyond the L1I (192, 128 and 160 KiB on Super, Performance and Efficiency cores) but within the L2. Each kernel is timed in two modes:

- **warm**: steady state over repeated calls, so at the larger sizes the predictor structures overflow;
- **cold**: before every timed call, a 2 MiB evictor region of plain random-direction branches and a 64 MiB data sweep flush the L1I, BTB, direction predictor, L2 and TLBs.

Each core type ran three passes of both the original and the swapped (`BC`-first) layout, with 15 warm and 101 cold repetitions per pass. Cold ratios are the medians of per-repetition paired ratios. Because placement effects flip between the layouts and a hint effect would not, each kernel's estimate is the geometric mean of its two layout means.

Table 3: Layout-cancelled `BC/B` ratio over the 19 cold-code kernels.

Core	Mode	Mean	SD	Min	Max
Super	warm	1.003	0.013	0.983	1.051
Super	cold	1.016	0.043	0.948	1.163
Performance	warm	1.002	0.008	0.996	1.035
Performance	cold	1.004	0.009	0.991	1.036
Efficiency	warm	1.000	0.009	0.970	1.020
Efficiency	cold	1.001	0.007	0.991	1.012

No kernel shows a `BC` benefit (Table 3). The largest deviations are the smallest Super-core cold kernels (4K–16K branches, about 30K cycles per call), where the ratio follows the layout. For example, tk-4096 measured 1.44 in the original layout and 0.94 in the swapped one; in both, the region placed second is slower. Eight kernels whose deviation kept the same sign in both layouts were rerun with eight more passes per layout and 151 cold repetitions. None replicated: for instance, Efficiency-core tk-524288 warm moved from 0.970 to 1.002/1.014.

The test reaches the regime where a hint could matter. In warm mix kernels, the direction predictor runs out of capacity between 256K and 512K branches on Super cores (1.9 to 8.4 cycles per branch) and between 64K and 256K branches on Performance and Efficiency cores (about 1.7 to 7.5). Mispredicts cannot be counted directly, but the cost is consistent with roughly half the branches mispredicting, even though every branch has a fixed direction. `BC` does not change these costs (0.995–1.008 at mix-524288 on every core).

5 Follow-up: random50, the taken-branch knee, and predictor pollution

Multi-placement control. A two-layout swap tests only two addresses. The decisive control (`gen_layout.py`, `layoutbench.c`) instead places 16 or 32 copies of each variant at different padding and gap offsets, with the variant order alternating between pairs. Every copy is measured in a shuffled order each round, and the reported statistic is the geometric mean of per-pair `BC/B` ratios with a t value across pairs. Placements are either a fixed progression (16 pairs) or random (32 pairs, two independent seeds).

random50. With 63 repetitions per run, Super and Efficiency cores give 0.999 in both layouts. Performance cores gave 0.850 in the original layout (twice) but 1.001 and 1.051 swapped. Across 16 placements, per-copy times range from 8.3 to 10.2 cycles per iteration, and the ratio is 1.010 ($t = 0.49$). The earlier outliers come from code placement.

Taken-branch knee. The tk sweep covered 21 sizes from 128 to 8192 branches.

- **Super cores:** the cost rises smoothly from 0.50 to 0.92 cycles per taken branch up to 4K branches, then jumps to 1.75 at 6K and 2.23 at 8K.
- **Performance cores:** 1.0 cycle per taken branch up to 320 branches, with a sharp knee between 320 and 384 and a plateau of 3.4 cycles from about 1.3K. Between 384 and 640 branches, whichever kernel sits second in memory is 20–47% slower: 1.47 originally and 0.66 swapped, cancelling to 1.000 in every pass.
- **Efficiency cores:** ratios swing by ±10–15% between 1152 and 3072 branches, with opposite signs at neighbouring sizes.

In the fixed-progression placement test, Super cores showed `BC` 3–5% faster at 1152–3072 branches (t between -1.5 and -2.6). With randomized placements, the same sizes gave 0.991–1.011 with mixed signs and $|t| < 1.8$ in both seeds. The apparent effect came from the regular placement pattern. Performance cores gave 1.000 at every size; Efficiency cores were noisy, with no $|t| > 1.4$.

Pollution. Each group of a pollution kernel holds a random branch A, immediately followed by a branch B on the same condition. B is predictable only from history, so it needs tagged predictor state. Both are `CBZ` and identical in the two variants, and are followed by F fixed-direction filler branches, the only code that differs. A matched control replaces B with two `ADD`s, so B's cost is (pol − polA) per group, measured separately for `B` and `BC` filler. B is learnable: alone it costs about 4 cycles per group, versus about 10 for the random A; with a few fillers between groups it falls to 0.7–2. For 1024 groups with $F \leq 64$ and 4096 groups with $F \leq 16$, the `BC`−`B` difference in B cost is within ±0.3 cycles per group on Performance cores, and is noise-dominated (±1–2) on Efficiency cores. On Super cores, 1024×64 favoured `BC` filler (-0.85, -0.57) but 4096×64 favoured `B` ($+0.34$, $+0.33$). With 256K filler branches, which overflow the direction predictor, A+filler costs 460–1830 cycles per group and the difference cannot be resolved. Whole-kernel `BC`/`B` ratios stayed within 0.994–1.018. The F=0 kernels, whose code is identical in both variants, measured up to 1.019 in one pass, which sets the noise floor.

6 Side observations

- All three core types sustain one not-taken conditional branch per cycle up to 16K static branches.
- Super cores sustain about 1.5 taken branches per cycle (0.68 cycles each) at 1K branches, rising to about 3.3–3.5 cycles per taken branch from 16K on.
- Performance cores reach about 3.2 cycles per taken branch already at 1K branches. Efficiency cores take about 1.2 cycles at 1K and 2.7 at 4K.
- Super cores run the alternating kernel at 1.0 cycle per iteration. Performance and Efficiency cores need 2.0 cycles.
- Not-taken branches still run at 1.0 cycle each from a 4 MiB footprint on Super cores (1.4 on Performance, 1.1 on Efficiency). Sequential fetch from L2 keeps up.
- Fully cold code (L2- and predictor-flushed) costs about 5–6 cycles per branch for not-taken branches, 8–9 for taken branches, about 10 for mixed directions, and 14–20 for the sparse layout.

7 Conclusion and limitations

The M6 reports FEAT_HBC and executes `BC.cond` correctly, with no undefined-instruction fault, but no core type shows a reproducible timing effect from the hint. That includes large-footprint and cold code, the case where a hint is most plausibly useful. Together with the absence of compiler support, this suggests that teaching compilers to emit `BC.cond` would currently bring no benefit on this machine.

The study has three limitations. First, it measures only cycles: without PMU event access, small changes in mispredicts or predictor allocation would be invisible. Second, the patterns tested are a small sample of possible behaviors. Third, it covers one microarchitecture under one OS release (macOS 27.0.1).

Artifacts. `~/opt/pmcrun/hbcbench/`: `gen_hbc.py`, `hbcbench.c`, `Makefile`, `run-coretypes.sh`, `repeat.sh`, `swap.sh`, `gen_cold.py`, `coldbench.c`, `cold-coretypes.sh`, `cold-repeat.sh`, `coldsum.py`, `gen_tab.py`, `tabbench.c`, `round3.sh`, `round3sum.py`, `gen_layout.py`, `layoutbench.c`, `layout-run.sh`, `layout-confirm.sh`, `build-main.sh`, `run-main.sh`, `mainsum.py`, and raw output in `coretypes.out`, `repeat.out`, `swap.out`, `cold-coretypes.out`, `cold-repeat.out`, `round3.out`, `layout-run.out`, `layout-confirm.out` and `run-main.out`.